

10. Key Concepts of Object-Oriented Design

Object-Oriented Design (OOD) is a software design methodology that models a software system as a collection of interacting objects.

Key Concepts of Object-Oriented Design

The fundamental concepts of Object-Oriented Design include:

1. The Central Role of Objects
2. The Notion of a Class
3. Abstract Specification of Functionality
4. A Language to Define the System (UML)
5. Standard Solutions (Design Patterns)
6. An Analysis Process to Model a System
7. The Notions of Extendability and Adaptability

1. The Central Role of Objects

The most important concept in Object-Oriented Design is the **object**. Unlike traditional software design, which focuses on processes or functions, Object-Oriented Design places **objects at the center of the software system**.

An object represents a real-world entity and contains both:

- **Data (Attributes)** – Information about the object.
- **Methods (Operations)** – Actions that can be performed on the object.

Characteristics

- Objects combine data and behavior.
- Objects represent real-world entities.
- Objects interact through messages (method calls).
- Objects make software modular and reusable.

Example: Bank Account Object

```
class BankAccount {
    private double balance;
    public void deposit(double amount) {
        balance += amount;
    }

    public void withdraw(double amount) {
        balance -= amount;
    }
}
```

Real-Time Example:

In an **Online Banking System**, objects include:

- Customer
- Account
- Transaction
- Loan
- ATM

Each object stores its own data and provides methods to manipulate that data.

2. The Notion of a Class

A **class** is a blueprint or template used to create objects. It defines the common attributes and methods shared by all objects of that type.

Classes allow software developers to classify objects into different categories and organize them into hierarchies.

Features

- Defines object structure.
- Supports classification.
- Enables inheritance.
- Supports specialization and generalization.

Example

```
class Vehicle {  
    String model;  
    void start() {  
        System.out.println("Vehicle Started");  
    }  
}  
  
Vehicle car = new Vehicle();  
Vehicle bike = new Vehicle();
```

3. Abstract Specification of Functionality

During software design, developers specify **what an object should do** without describing **how it performs the task**. This abstract specification is called an **interface** or an **abstract class**.

Characteristics

- Defines functionality.
- Hides implementation details.
- Supports abstraction.
- Promotes loose coupling.

Example (Interface)

```
interface Payment {  
    void pay(double amount);  
}
```

Different classes implement the interface.

```
class CreditCard implements Payment {  
    public void pay(double amount) {  
        System.out.println("Paid using Credit Card");  
    }  
}
```

Real-Time Example: ATM Machine

The customer knows **what operations** are available:

- Withdraw
- Deposit
- Balance Inquiry

The customer does not know **how** these operations are implemented.

4. A Language to Define the System (UML)

The **Unified Modeling Language (UML)** is the standard graphical language used to describe object-oriented software systems.

Purpose

- Describe system structure.
- Describe system behavior.
- Communicate designs.
- Document software architecture.

Common UML Diagrams

- Class Diagram
- Object Diagram
- Use Case Diagram
- Sequence Diagram
- Activity Diagram
- State Diagram
- Component Diagram
- Deployment Diagram

Example

Simple UML Class Diagram

```
+-----+
|      Student      |
+-----+
| - id              |
| - name            |
+-----+
| + display()       |
+-----+
```

Advantages

- Standard notation.
- Easy communication among developers.
- Better documentation.
- Simplifies software design.

5. Standard Solutions (Design Patterns)

A **Design Pattern** is a proven, reusable solution to a commonly occurring software design problem. Design patterns provide standard ways of organizing classes and objects.

Examples

Problem	Design Pattern
One object only	Singleton
Flexible object creation	Factory Method
Add functionality dynamically	Decorator
Convert incompatible interfaces	Adapter
Notify multiple objects	Observer

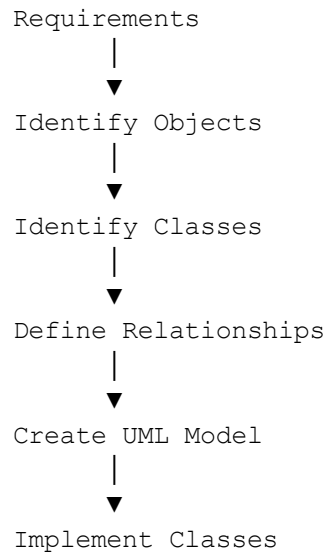
Advantages

- Reusable solutions.
- Faster software development.
- Better maintainability.
- Standard software architecture.

6. An Analysis Process to Model a System

Object-Oriented Design provides a **systematic analysis process** for converting software requirements into an object-oriented design. This process identifies the conceptual classes required by the system and transforms them into implementation classes.

Development Process



Example: Library Management System

Conceptual Classes:

- Book
- Student
- Librarian
- Issue
- Fine

Implementation Classes:

- o Book
- o Student
- o Library
- o IssueBook
- o FineCalculator

Advantages

- Systematic development.
- Better requirement analysis.
- Easy transition from design to implementation.

7. Extendability and Adaptability

Object-Oriented Design allows existing software to be **extended** and **adapted** without rewriting the entire system. Two important mechanisms support this:

- Inheritance
- Composition

A. Inheritance

Inheritance creates a new class from an existing class.

Example

```
class Vehicle{  
  
}  
  
class Car extends Vehicle{  
  
}
```

Benefits

- Code reuse.
- Easy extension.
- Supports specialization.

B. Composition

Composition builds a new class using objects of existing classes.

Example

```
class Engine{  
  
}  
  
class Car{  
  
    private Engine engine = new Engine();  
  
}
```

Benefits

- High flexibility.
- Loose coupling.
- Better maintainability.

Real-Time Example: Smartphone

A smartphone consists of:

- Camera
- Battery
- Processor

- Display

These components work together to form a complete smartphone.